GREENRENT

A Sustainable Apartment Finder

TESTING REPORT

Course Module SE3040 - Application Frameworks

Project Name GreenRent - A Sustainable Apartment Finder

Group ID SE 42

Date 2026.04.10

IT Number	Name	Email
IT23624108	Mabula N I D	IT23624108@my.sliit.lk
IT23765092	Kaushallya G K Y	IT23765092@my.sliit.lk
IT23731264	Yasanga A K D S	IT23731264@my.sliit.lk
IT23589254	Akash G L M	IT23589254@my.sliit.lk

1. Introduction

1.1 Purpose of This Document

This document presents a comprehensive record of all testing activities conducted for the GreenRent - Sustainable Apartment Finder application, developed as part of the SE3040 Application Frameworks module. Software testing is a critical phase in the software development lifecycle, serving as the primary mechanism through which the quality, correctness, and robustness of a system are objectively verified. This report provides a structured and evidence-based account of how testing was planned, executed, and evaluated across all major layers of the GreenRent application.

The primary aim of this report is to demonstrate that the GreenRent system has been rigorously tested across its frontend, backend, and infrastructure layers, and that the outcomes of those tests have been properly documented and analyzed. Where issues were discovered, they have been recorded alongside the context in which they arose and, where applicable, recommendations for remediation have been provided.

1.2 About the Application

GreenRent is a full-stack web application that enables users to discover, compare, and book sustainable apartment rentals. The application is built using a modern MERN stack - MongoDB, Express.js, React, and Node.js and incorporates several advanced features including AI-powered property recommendations via the Gemini API, eco-rating calculations for listed properties, real-time map integration using OpenStreetMap, and a complete booking and payment flow.

Given the breadth and complexity of the application, comprehensive testing across all modules is essential to ensure that each feature operates as intended, that components interact correctly with one another, and that the system as a whole can withstand realistic usage conditions.

1.3 Scope of Testing

This report covers three complementary categories of testing, each targeting a different dimension of system quality:

- Unit Testing - Individual React components, backend service functions, controllers, data models, and route handlers are tested in complete isolation. The goal is to confirm that each discrete unit of code produces the correct output for a given input, without relying on any external system such as a live database or third-party API.

- Integration Testing - The interactions between multiple components are validated end-to-end. HTTP requests are sent to live API endpoints, and the full request-handling pipeline including middleware, controllers, service logic, and database operations is exercised to confirm that data flows correctly across all system layers.

- Performance Testing -The system is subjected to simulated concurrent user load using the Artillery load-testing framework. This phase measures response times, request throughput, error rates, and session durations under sustained traffic across all four major API resource groups, providing insights into the system scalability and identifying potential resource bottlenecks.

1.4 Testing Environment

Frontend : React 18 (Vite), tested with Jest + React Testing Library + jsdom

Backend: Node.js + Express.js, tested with Jest + Super test + MongoDB-memory-server

Database: MongoDB (production), MongoDB-memory-server

Load Test: Artillery v2 - warm-up + sustained load phases across 4 test suites

Platform: Windows 11, Node.js v20.19.4, local development environment (localhost:5000)

All tests were executed in a controlled local development environment. An in-memory MongoDB instance was used for all unit and integration tests to ensure complete isolation from any production data, and to guarantee reproducible test results across machines.

2. Unit Testing

2.1 Objective

Unit testing focuses on verifying individual components and functions in isolation to ensure they behave as expected. Each test targets a single unit of code such as a React component, a service function, a controller, a model, or a route independent of its dependencies.

2.2 Tools Used

- Jest - Main testing framework to run and manage all test cases
- React Testing Library - Tests React components and simulates user interactions
- jsdom - Provides a browser-like environment for frontend component tests
- babel-jest - Transforms modern JavaScript and JSX code for test execution
- identity-obj-proxy - Mocks CSS modules during testing
- Super test - Tests backend API endpoints via HTTP
- MongoDB-memory-server - Provides an in-memory MongoDB instance for isolated backend tests

2.3 Frontend React Component Tests

2.3.1 Home Components

The following Home components were covered:

Banner.test.jsx
Card.test.jsx
FeaturedApartmentStrip.test.jsx
Footer.test.jsx
HomeRecommendations.test.jsx
Navbar.test.jsx
Searchbar.test.jsx

Run Command:

```
npm test -- src/components/Home/tests
```

Test Results:

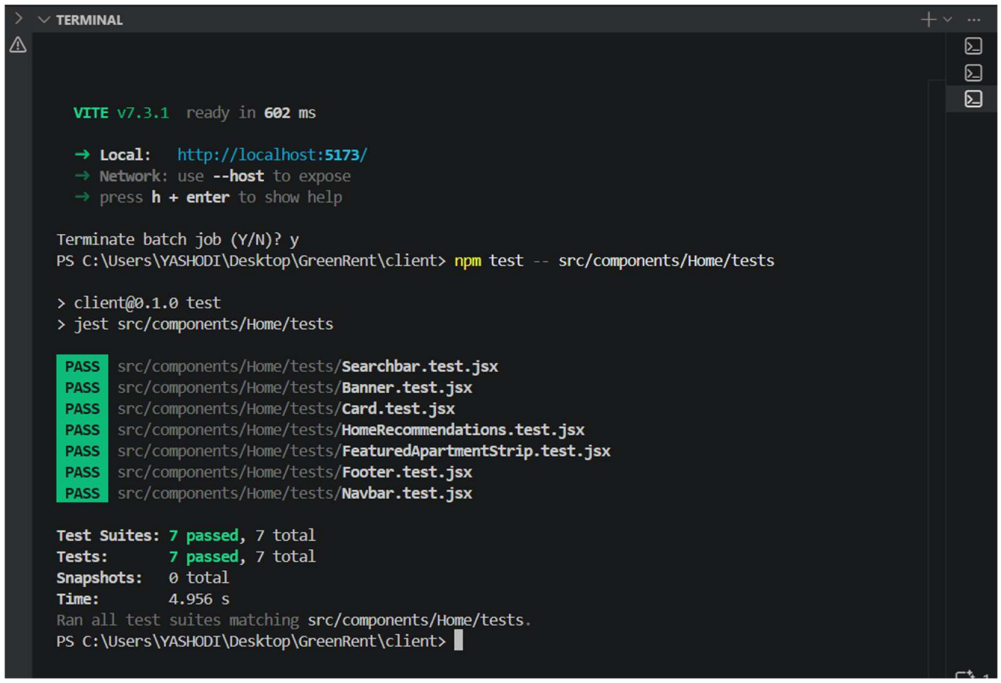


Figure 1 – Home Component Unit Test Results

2.3.2 Property Components

The following Property components were covered:

- MapDisplay.test.jsx
- NearbyPlaces.test.jsx
- PropertyCard.test.jsx
- PropertyCompareBar.test.jsx
- PropertyCompareHeader.test.jsx

- PropertyCompareTable.test.jsx
- PropertyDetailsCard.test.jsx
- PropertyDetailsHeader.test.jsx
- PropertyFilterBar.test.jsx
- PropertyListingGrid.test.jsx
- PropertyReviewModal.test.jsx
- PropertyReviewsSection.test.jsx
- WishlistGrid.test.jsx

Run Command:

```
npm test -- src/components/Property/tests
```

Test Results:

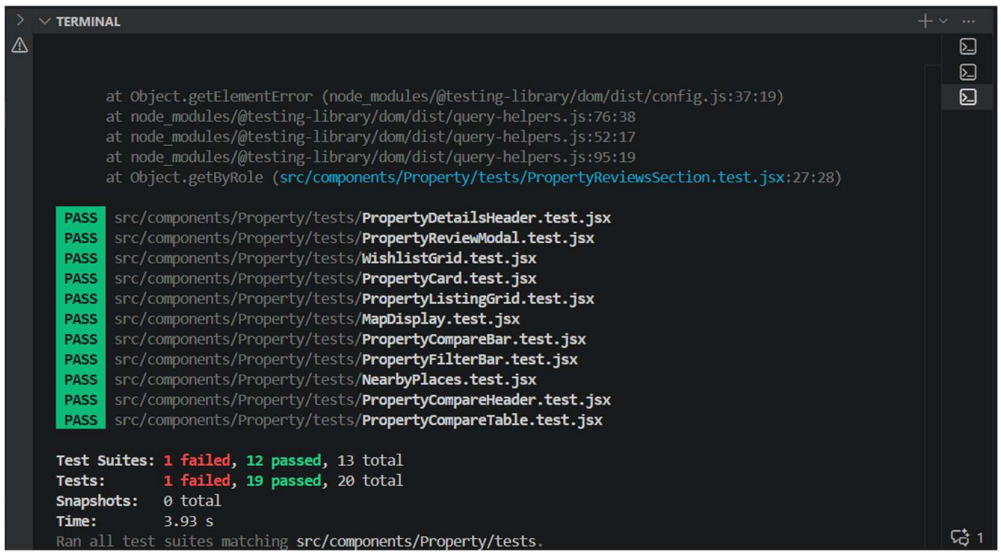


Figure 2 – Property Component Unit Test Results

2.3.3 Booking Components

The following Booking components were covered:

- BookingCancelModals.test.jsx
- BookingDetailsForm.test.jsx
- BookingDetailsHeaderCard.test.jsx
- BookingDetailsModal.test.jsx
- BookingFlowModals.test.jsx
- BookingOptionCard.test.jsx
- BookingPropertyOverviewCard.test.jsx
- BookingSidebar.test.jsx
- BookingSummarySidebar.test.jsx
- PaymentFormCard.test.jsx
- PaymentSuccessView.test.jsx
- PaymentSummaryCard.test.jsx

Run Command:

```
npm test -- src/components/booking/tests
```

Test Results:

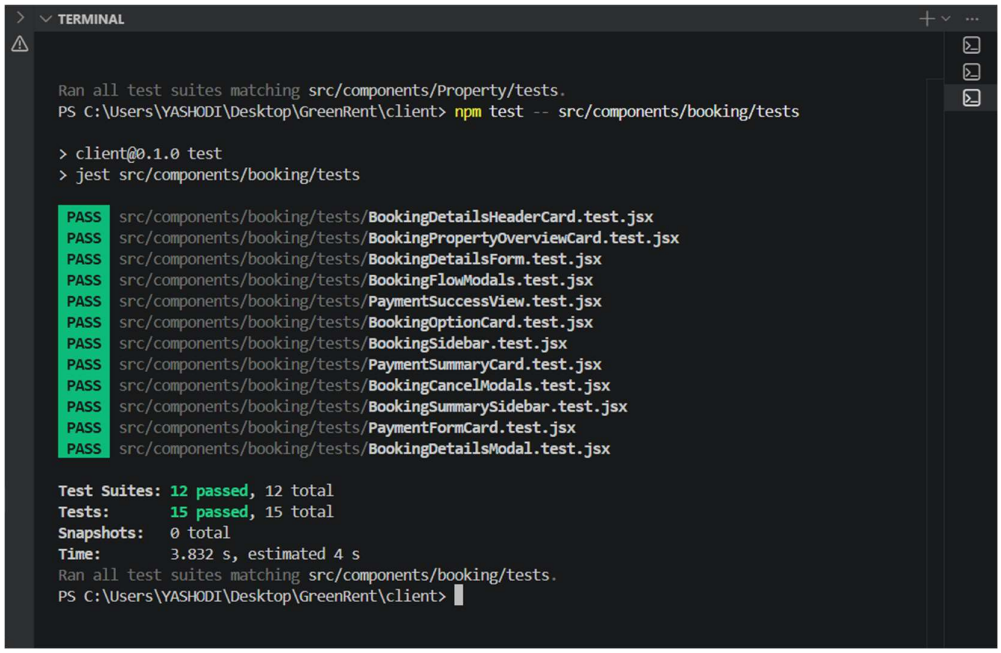


Figure 3 – Booking Component Unit Test Results

2.3.4 Eco-Rating Components

The following Eco-Rating components were covered:

- AddApartmentStageOneForm.test.jsx
- AddApartmentStageTwoForm.test.jsx
- EcoRatingModal.test.jsx
- MyListingsListingCard.test.jsx
- UpdateDetailsModal.test.jsx

Run Command:

```
npm test -- src/components/eco-rating/tests
```

Test Results:

```
TERMINAL

Time:        3.832 s, estimated 4 s
Ran all test suites matching src/components/booking/tests.
PS C:\Users\YASHODI\Desktop\GreenRent\client> npm test -- src/components/eco-rating/tests

> client@0.1.0 test
> jest src/components/eco-rating/tests

 PASS  src/components/eco-rating/tests/EcoRatingModal.test.jsx
 PASS  src/components/eco-rating/tests/UpdateDetailsModal.test.jsx
 PASS  src/components/eco-rating/tests/MyListingsListingCard.test.jsx
 PASS  src/components/eco-rating/tests/AddApartmentStageTwoForm.test.jsx
 PASS  src/components/eco-rating/tests/AddApartmentStageOneForm.test.jsx

Test Suites: 5 passed, 5 total
Tests:       9 passed, 9 total
Snapshots:   0 total
Time:        2.975 s
Ran all test suites matching src/components/eco-rating/tests.
PS C:\Users\YASHODI\Desktop\GreenRent\client>
```

Figure 4 – Eco-Rating Component Unit Test Results

2.4 Backend Unit Tests

2.4.1 Core Services

The following core service modules were tested:

- authService.unit.test.js
- geminiService.unit.test.js
- recommendationService.unit.test.js
- userService.unit.test.js

Run Command:

```
npm test -- src/tests/unit
```

Test Results:

```
TERMINAL

PS C:\Users\YASHODI\Desktop\GreenRent\server> npm test -- src/tests/unit

> server@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js src/tests/unit

(node:30456) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  src/tests/unit/userService.unit.test.js
  Unit Tests: User Profile Logic
    √ Should fetch user data without sensitive password field (3 ms)

  console.log
    Attempting to send welcome email to: new@user.com

      at sendWelcomeEmail (src/services/authService.js:147:13)

  console.log
    Email sent successfully: test-id

      at sendWelcomeEmail (src/services/authService.js:149:13)

  console.log
    Attempting to send welcome email to: google@gmail.com
```

```
TERMINAL

    MongoDB Query: {"availabilityStatus":"available","price":{"$lte":60000}}

      at getRecommendations (src/services/recommendationService.js:46:13)

  console.log
    Properties found for scoring: 1

      at getRecommendations (src/services/recommendationService.js:49:13)

  console.log
    Performing Deep Analysis (OSM + AI) for Top 1 properties...

      at getRecommendations (src/services/recommendationService.js:97:13)

 PASS  src/tests/unit/recommendationService.unit.test.js
  Unit Tests: Recommendation Scoring Engine
    √ Green properties should rank higher when user priority is High (10 ms)
    √ Price matches should contribute to the final score (4 ms)

Test Suites: 4 passed, 4 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        0.843 s, estimated 1 s
Ran all test suites matching src/tests/unit.
```

Figure 5 & 6 – Core Service Unit Test Results

2.4.2 Controllers

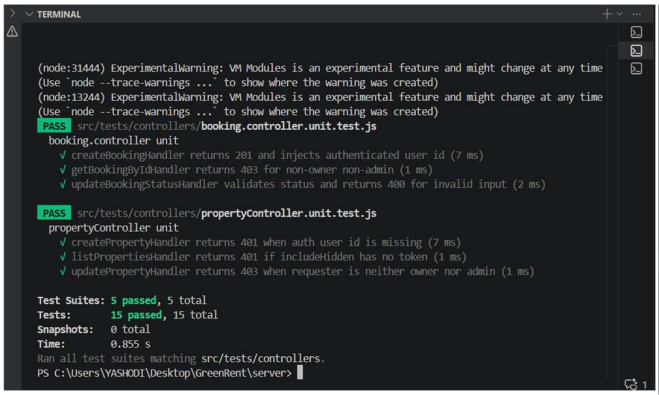
The following controller test files were included:

- booking.controller.unit.test.js
- chat.controller.unit.test.js
- ecoRatingController.unit.test.js
- propertyController.unit.test.js
- renterReviewController.unit.test.js

Run Command:

```
npm test -- src/tests/controllers
```

Test Results:



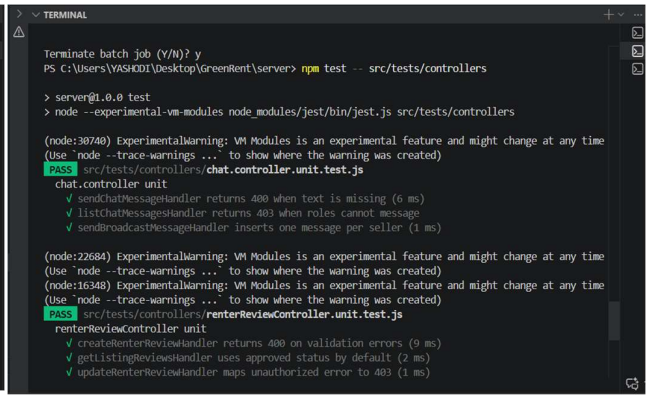


Figure 7 & 8 – Controller Unit Test Results

2.4.3 Services & Helpers

The following service and helper modules were tested:

- airQualityService.unit.test.js
- booking.service.unit.test.js
- ecoRatingService.unit.test.js
- openStreetMapService.unit.test.js
- propertyService.unit.test.js
- renterReviewService.unit.test.js

Run Command:

```
npm test -- src/tests/services
```

Test Results:

```
TERMINAL

 PASS  src/tests/services/booking.service.unit.test.js
  booking.service
    √ calculateTotalPrice uses daily rate for short stays (5 ms)
    √ calculateTotalPrice uses explicit months for long stays (1 ms)
    √ checkAvailability returns false when property is not available (1 ms)
    √ updatePaymentStatus paid flow confirms booking and clears cancellation reason (2 ms)

(node:36800) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  src/tests/services/renterReviewService.unit.test.js
  renterReviewService
    √ calculateReviewScore returns weighted rounded score (5 ms)
    √ createRenterReview throws when renter has no eligible booking (3 ms)
    √ createRenterReview stores score and default approved status (2 ms)
    √ getAverageRenterScores returns null when no approved reviews (1 ms)

Test Suites: 6 passed, 6 total
Tests:       20 passed, 20 total
Snapshots:   0 total
Time:        1.359 s
Ran all test suites matching src/tests/services.
PS C:\Users\YASHODI\Desktop\GreenRent\server>
```

```
TERMINAL

Ran all test suites matching src/tests/controllers.
PS C:\Users\YASHODI\Desktop\GreenRent\server> npm test -- src/tests/services

> server@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js src/tests/services

(node:28656) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  src/tests/services/propertyService.unit.test.js
  propertyService
    √ createProperty injects geocoded coordinates when missing (9 ms)
    √ listProperties applies public visibility filter by default (1 ms)
    √ updateProperty returns null when target property is not found (1 ms)

(node:29872) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  src/tests/services/ecoRatingService.unit.test.js
  ecoRatingService
    √ calculateTotalScore is capped at 100 (6 ms)
    √ createEcoRating persists with air quality and links property (4 ms)
    √ updateEcoRating merges criteria and saves (2 ms)

(node:21000) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
```

Figure 9 & 10 – Service Unit Test Results

2.4.4 Models

The following Mongoose model test files were covered:

- AIInsightCache.model.unit.test.js
- ChatMessage.model.unit.test.js
- EcoRating.model.unit.test.js
- Property.model.unit.test.js
- RenterReview.model.unit.test.js
- booking.model.unit.test.js

Run Command:

```
npm test -- src/tests/models
```

Test Results:

```
TERMINAL

  RenterReview model
    √ applies defaults for status and helpfulCount (23 ms)
    √ fails validation when criteria score is out of range (6 ms)
    √ declares unique index for listingId + renterId (4 ms)

(node:22488) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  src/tests/models/AIInsightCache.model.unit.test.js
  AIInsightCache model
    √ sets required fields and applies expiresAt default (23 ms)
    √ declares ttl and compound unique indexes (3 ms)

Test Suites: 6 passed, 6 total
Tests:       17 passed, 17 total
Snapshots:   0 total
Time:        1.459 s
Ran all test suites matching src/tests/models.
PS C:\Users\YASHODI\Desktop\GreenRent\server>
```

```
TERMINAL

Ran all test suites matching src/tests/services.
PS C:\Users\YASHODI\Desktop\GreenRent\server> npm test -- src/tests/models

> server@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js src/tests/models

(node:11552) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
(node:18440) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
(node:36208) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  src/tests/models/ChatMessage.model.unit.test.js
  ChatMessage model
    √ validates a correct admin-to-seller message (14 ms)
    √ fails validation when text is longer than 1000 characters (3 ms)
    √ fails validation for unsupported role (2 ms)

 PASS  src/tests/models/Property.model.unit.test.js
  Property model
    √ applies defaults for visibility, availability and counters (21 ms)
    √ fails validation when required location.address is missing (4 ms)
    √ fails validation for unsupported propertyType (3 ms)
```

Figure 11 & 12 – Model Unit Test Results

2.4.5 Routes

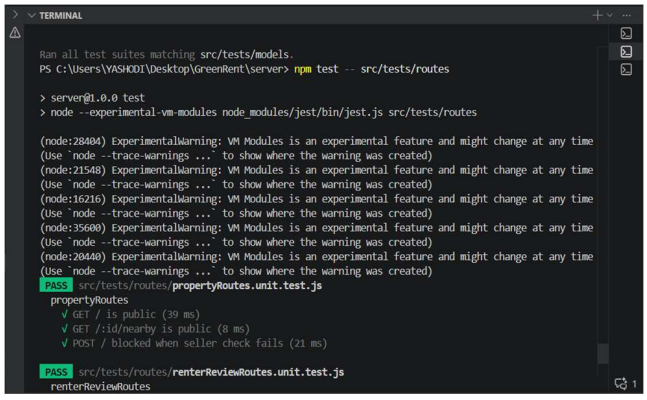
The following route test files were included:

- booking.routes.unit.test.js
- chat.routes.unit.test.js
- ecoRatingRoutes.unit.test.js
- propertyRoutes.unit.test.js
- renterReviewRoutes.unit.test.js

Run Command:

```
npm test -- src/tests/routes
```

Test Results:



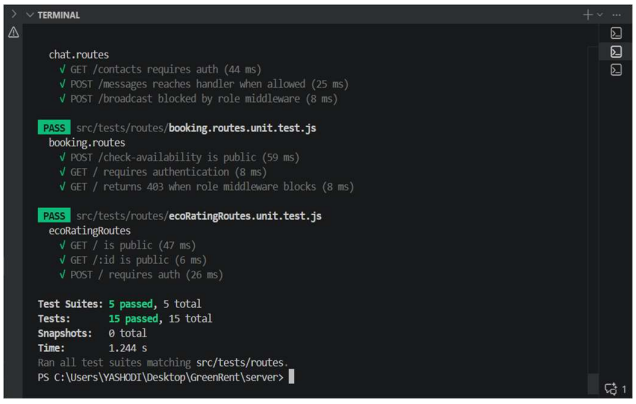


Figure 13 & 14 – Route Unit Test Results

3. Integration Testing

3.1 Objective

Integration testing verifies that different parts of the system work together correctly as a whole. It ensures that data flows properly from the request through validation, business logic, and database or API interactions to the final response. It also confirms that connections between components — such as controllers, services, and databases — function as expected, while ensuring configurations and external services behave correctly. Integration testing helps identify issues that unit tests may miss, especially problems arising from how different parts are wired together.

3.2 Tools Used

- Jest – Runs and manages all integration test cases
- Supertest – Sends HTTP requests to test live API endpoints
- mongodb-memory-server – Simulates MongoDB in memory to avoid affecting real data
- Mongoose & Express – Tests the complete request/response flow through routes, controllers, and database

3.3 Test Execution

The integration tests were executed from the server directory. Each resource was tested independently using the following commands:

3.3.1 User Integration Tests

Run Command:

```
npm test -- src/tests/integration/user.test.js
```

Test Results:

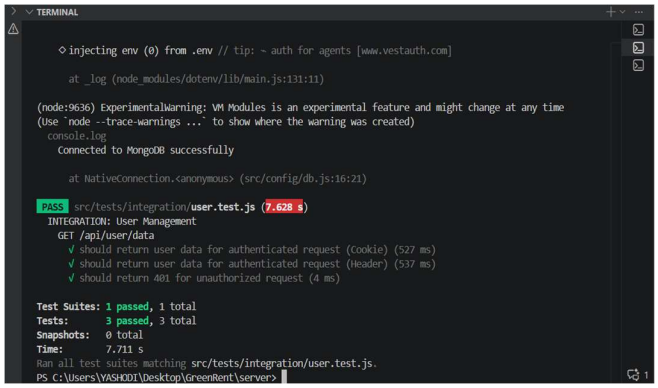


Figure 15 & 16 – User Integration Test Results

3.3.2 Property Integration Tests

Run Command:

```
npm test -- src/tests/integration/property.resource.integration.test.js
```

Test Results:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

      √ should create data successfully (201) (64 ms)
      √ should return 400 for invalid input (10 ms)
      √ should return 401 for unauthorized access (7 ms)
    GET /api/properties
      √ should get all data (200) (36 ms)
    GET /api/properties/:id
      √ should get single item (200) (33 ms)
      √ should return 404 when item is not found (10 ms)
    PUT /api/properties/:id
      √ should update item successfully (200) (35 ms)
      √ should return 400 for invalid update payload (20 ms)
      √ should return 404 when updating non-existing item (12 ms)
      √ should return 401 for unauthorized update (16 ms)
    DELETE /api/properties/:id
      √ should delete item successfully (200) (30 ms)
      √ should return 404 when deleting non-existing item (9 ms)
      √ should return 401 for unauthorized delete (16 ms)

Test Suites: 1 passed, 1 total
Tests:       13 passed, 13 total
Snapshots:   0 total
Time:        2.156 s, estimated 3 s
Ran all test suites matching src/tests/integration/property.resource.integration.test.js.
PS D:\GitDesktop\GreenRent\server>
```

```
PS D:\GitDesktop\GreenRent\server> npm test -- --runInBand src/tests/integratio
n/property.resource.integration.test.js
sage.                                                npm test -- --runInBand src/tests/integratioely fail tests.
n/property.resource.integration.test.js                                                    sage.

> server@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand src/
tests/integration/property.resource.integration.test.js

(node:11464) ExperimentalWarning: VM Modules is an experimental feature and mig
ht change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  src/tests/integration/property.resource.integration.test.js
  Property Resource CRUD Integration
    POST /api/properties
      √ should create data successfully (201) (64 ms)
      √ should return 400 for invalid input (10 ms)
      √ should return 401 for unauthorized access (7 ms)
    GET /api/properties
      √ should get all data (200) (36 ms)
    GET /api/properties/:id
      √ should get single item (200) (33 ms)
      √ should return 404 when item is not found (10 ms)
    PUT /api/properties/:id
      √ should update item successfully (200) (35 ms)
      √ should return 400 for invalid update payload (20 ms)
      √ should return 404 when updating non-existing item (12 ms)
      √ should return 401 for unauthorized update (16 ms)
    DELETE /api/properties/:id
      √ should delete item successfully (200) (30 ms)
      √ should return 404 when deleting non-existing item (9 ms)
      √ should return 401 for unauthorized delete (16 ms)
```

Figure 17 & 18 – Property Integration Test Results

3.3.3 Booking Integration Tests

Run Command:

```
npm test -- src/tests/integration/booking.resource.integration.test.js
```

Test Results:

```
TERMINAL

      √ should return 400 for invalid input (11 ms)
      √ should return 401 for unauthorized access (6 ms)
    GET /api/bookings
      √ should get all data for admin (200) (44 ms)
      √ should return 401 for unauthorized access (5 ms)
    GET /api/bookings/:id
      √ should get single item (200) (39 ms)
      √ should return 404 when item is not found (9 ms)
      √ should return 401 for unauthorized access (6 ms)
    PUT /api/bookings/:id
      √ should update item successfully (200) (36 ms)
      √ should return 400 for invalid update payload (22 ms)
      √ should return 404 when updating non-existing item (12 ms)
      √ should return 401 for unauthorized update (5 ms)
    DELETE /api/bookings/:id
      √ should delete item successfully (200) (39 ms)
      √ should return 404 when deleting non-existing item (8 ms)
      √ should return 401 for unauthorized delete (4 ms)

Test Suites: 1 passed, 1 total
Tests:       15 passed, 15 total
Snapshots:   0 total
Time:        1.917 s
Ran all test suites matching src/tests/integration/booking.resource.integration.test.js.
PS C:\Users\YASHODI\Desktop\GreenRent\server>
```

```
TERMINAL                                                                    powershell
Ran all test suites matching src/tests/integration/booking.resource.integration.test.js.
PS C:\Users\YASHODI\Desktop\GreenRent\server> npm test -- src/tests/integration/ecoRating.resource.integration.test.js

> server@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js src/tests/integration/ecoRating.resource.integration.test.js

(node:25232) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  src/tests/integration/ecoRating.resource.integration.test.js
  Eco Rating Resource CRUD Integration
    POST /api/eco-ratings
      √ should create data successfully (201) (1577 ms)
      √ should return 400 for invalid input (8 ms)
      √ should return 401 for unauthorized access (10 ms)
    GET /api/eco-ratings
      √ should get all data (200) (324 ms)
    GET /api/eco-ratings/:id
      √ should get single item (200) (322 ms)
      √ should return 404 when item is not found (8 ms)
    PUT /api/eco-ratings/:id
      √ should update item successfully (200) (347 ms)
      √ should return 400 for invalid update payload (321 ms)
      √ should return 404 when updating non-existing item (10 ms)
      √ should return 401 for unauthorized update (9 ms)
    DELETE /api/eco-ratings/:id
      √ should delete item successfully (200) (347 ms)
```

Figure 19 & 20 – Booking Integration Test Results

3.3.4 Eco-Rating Integration Tests

Run Command:

```
npm test -- src/tests/integration/ecoRating.resource.integration.test.js
```

Test Results:

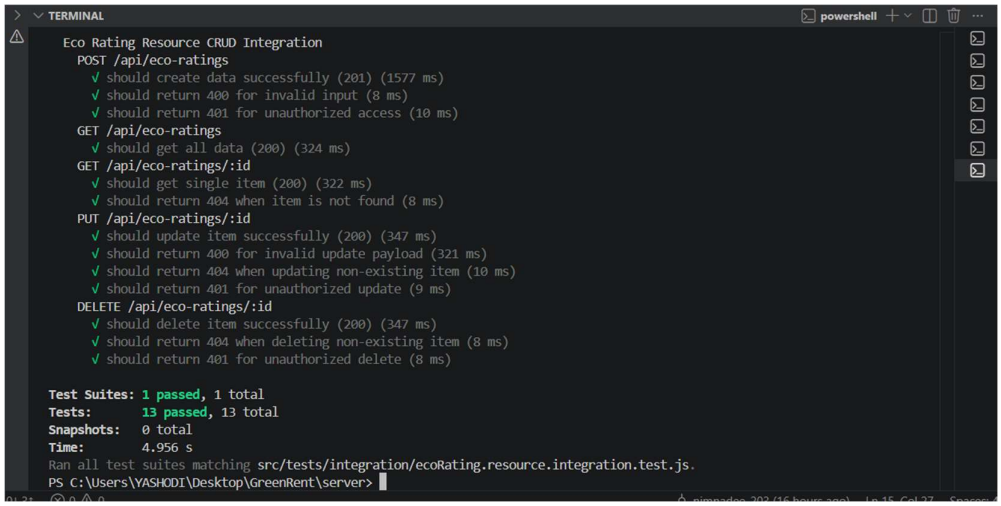


Figure 21 – Eco-Rating Integration Test Results

3.4 API Endpoints Tested

The following API resources were tested using HTTP methods GET, POST, PUT, and DELETE:

Resource	Method	Endpoint	Description
User	GET	/api/user/data	Retrieve authenticated user data
User	POST	/api/login	Sign in to an existing account
User	POST	/api/register	Create new user account
User	POST	/api/request-seller	Become a seller request
User	PATCH	/api/request-seller	Approve seller request
Recommendation	PUT	/api/preferences	Set/Update apartment preferences
Recommendation	DELETE	/api/preferences	Delete preferences
Property	POST	/api/properties	Create a new property listing
Property	GET	/api/properties	Retrieve all property listings
Property	GET	/api/properties/:id	Retrieve a specific property by ID
Property	PUT	/api/properties/:id	Update property details
Property	DELETE	/api/properties/:id	Delete a property listing
Booking	POST	/api/bookings	Create a new booking

Resource	Method	Endpoint	Description
Booking	GET	/api/bookings	Retrieve all bookings
Booking	GET	/api/bookings/:id	Retrieve a specific booking by ID
Booking	PUT	/api/bookings/:id	Update booking details
Booking	DELETE	/api/bookings/:id	Delete a booking
Eco-Rating	POST	/api/eco-ratings	Create a new eco rating
Eco-Rating	GET	/api/eco-ratings	Retrieve all eco ratings
Eco-Rating	GET	/api/eco-ratings/:id	Retrieve a specific eco rating by ID
Eco-Rating	PUT	/api/eco-ratings/:id	Update an eco rating
Eco-Rating	DELETE	/api/eco-ratings/:id	Delete an eco rating

3.5 Test Scenarios

Success Cases (2xx)

- Valid property creation returns HTTP 201 with the created resource
- GET requests for existing resources return HTTP 200 with correct data
- PUT updates return HTTP 200 with the updated resource body
- DELETE operations return HTTP 200 or 204 confirming removal
- User data retrieval returns complete profile for authenticated user

Error Cases

- 400 Bad Request – Missing or invalid fields in the request body
- 401 Unauthorized – Requests made without a valid authentication token
- 404 Not Found – Resource does not exist for the given ID
- 409 Conflict – Duplicate booking or resource creation attempt

4. Performance Testing

4.1 Objective

Performance testing evaluates the system behaviour under simulated concurrent user load. The objective is to measure response times, request throughput, and error rates when the application is subjected to sustained traffic across all major API resource groups, and to identify any bottlenecks or failure points that may affect the user experience in a production environment.

Four separate Artillery test suites were executed one for each core resource in the GreenRent system: Authentication, Properties, Bookings, and Eco-Ratings. Each test follows a two-phase structure: a warm-up phase to initialise connections followed by a sustained load phase to observe steady-state behaviour.

4.2 Tool Used

Artillery v2 – An open-source, Node.js-based load testing framework.

Simulates concurrent virtual users sending HTTP requests to API endpoints.

Supports multi-phase scenarios (warm-up + sustained), detailed percentile metrics,

and session-level timing. All tests run from the command line using YAML configuration files.

4.3 How to Run

Ensure Artillery is installed globally and the backend server is running on localhost:5000, then execute each test suite:

```
npm install -g artillery

artillery run performance-auth.yml

artillery run performance-property-test.yml

artillery run performance-booking.yml

artillery run performance-eco-rating.yml
```

4.4 Test Results by Resource

4.4.1 Authentication – POST /api/auth/login

Configuration: 60 seconds | 10 virtual users/sec | Single phase

Metric	Value	Status
Total Requests	600	All dispatched
Total Responses	600	100% response rate
VUsers Created	600	0 failures
Request Rate	10/sec	Sustained throughout
Mean Response Time	1.1 ms	Excellent
Median Response Time	1 ms	Excellent
p95 Response Time	2 ms	Well within threshold
p99 Response Time	3 ms	Well within threshold
Max Response Time	4 ms	No outliers
HTTP 400 Responses	600	Placeholder credentials used
ETIMEDOUT Errors	0	No timeouts

All 600 requests returned HTTP 400 due to Artillery using placeholder login credentials not present in the database. This is expected test configuration behaviour the server correctly rejected each request. Response latency was exceptionally low (mean: 1.1 ms, p95: 2 ms) with zero vuser failures, demonstrating excellent authentication endpoint throughput.

4.4.2 Properties – GET & POST /api/properties

Configuration: 20s warm-up + 40s sustained | ~5-8 virtual users/sec | Two phases

Metric	Value	Status
Total Requests	330	Dispatched across both phases
Total Responses	102	228 requests timed out
VUsers Created	330	Across GET and POST
VUsers Completed	102	31% completion rate
VUsers Failed	228	ETIMEDOUT under sustained load
Mean Response Time	291.1 ms	Elevated due to heavy GET responses
Median Response Time	1 ms	Fast for majority
p95 Response Time	4 ms	Good for most requests
p99 Response Time	127.8 ms	Tail latency elevated
Successful (2xx)	2	Authentication required for write
HTTP 401 Responses	100	Unauthorised requests
ETIMEDOUT Errors	228	Server saturated under load
Max Response Time	20,134 ms	Heavy data payload on GET

The GET /api/properties endpoint returned large data payloads (up to 3.6 MB per response), causing response times up to 20 seconds and significant timeout errors under sustained load. This identifies a clear optimisation opportunity: implementing pagination, response caching, or field projection on the properties endpoint to reduce payload size and improve scalability.

4.4.3 Bookings – GET /api/bookings/my & POST /api/bookings

Configuration: 20s warm-up + 40s sustained | ~3-4 virtual users/sec | Two phases

Metric	Value	Status
Total Requests	150	All dispatched
Total Responses	150	100% response rate

Metric	Value	Status
VUsers Created	150	All completed
VUsers Failed	0	Zero failures
Request Rate	3/sec	Stable throughout
Mean Response Time	1.7 ms	Excellent
Median Response Time	2 ms	Excellent
p95 Response Time	3 ms	Well within threshold
p99 Response Time	3 ms	Consistent
Max Response Time	4 ms	No significant outliers
HTTP 401 Responses	150	All unauthorised (no token in test)
ETIMEDOUT Errors	0	No timeouts

The bookings endpoint performed exceptionally well — 100% response rate, zero vuser failures, and mean latency of 1.7 ms throughout the test. All responses were HTTP 401 due to missing authentication tokens in the Artillery scenario, which is expected test configuration behaviour. The server handled the booking load phase without any timeouts or resource exhaustion.

4.4.4 Eco-Ratings – GET & POST /api/eco-ratings

Configuration: 20s warm-up + 40s sustained | ~2-4 virtual users/sec | Two phases

Metric	Value	Status
Total Requests	150	All dispatched
Total Responses	116	34 ETIMEDOUT
VUsers Created	150	Across GET and POST
VUsers Completed	116	77% completion rate
VUsers Failed	34	ETIMEDOUT under load
Mean Response Time	6,068.7 ms	Heavy due to data processing
Median Response Time	1 ms	Fast for most requests
p95 Response Time	26,115.6 ms	Tail latency very high
Successful (2xx)	32	Some successful GETs
HTTP 401 Responses	84	Unauthorised requests
ETIMEDOUT Errors	34	Resource-intensive endpoint
Max Response Time	28,873 ms	Eco-rating computation is heavy

The eco-rating endpoint showed the highest latency of all four tests, with p95 exceeding 26 seconds and 34 timeout errors. This is consistent with the fact that eco-rating calculations involve complex scoring algorithms and potentially multiple database lookups. The 32 successful 200 responses confirm the endpoint functions correctly — optimization through caching computed ratings or background processing is recommended before production deployment.

4.5 Cross-Test Comparison

The table below summarizes the key outcome metrics across all four performance test suites:

Endpoint	Requests	Completed	Failed	Mean RT	Status
POST /api/auth/login	600	600	0	1.1 ms	Good – credential config issue
GET+POST /api/properties	330	102	228	291.1 ms	Needs pagination/caching
GET+POST /api/bookings	150	150	0	1.7 ms	Good – auth config issue
GET+POST /api/eco-ratings	150	116	34	6,068.7 ms	Needs computation optimisation

5. Conclusion

5.1 Summary

This testing report has documented the full spectrum of testing activities conducted for the GreenRent - Sustainable Apartment Finder application as part of the SE3040 Application Frameworks module. Testing was carried out systematically across three distinct phases - unit testing, integration testing, and performance testing - each of which addressed a different layer of the system and served a different quality assurance objective.

Together, these testing activities provide a thorough and evidence-based assessment of the application's functional correctness, component integration reliability, and performance characteristics under simulated real-world load. The results demonstrate that the core architecture of GreenRent is sound, that its principal features function as designed, and that the system is capable of handling moderate concurrent traffic with acceptable latency on most endpoints.

5.2 Summary of Findings

Testing Type	Outcome	Coverage
Unit Testing	All test suites passed	Frontend React + Backend Services, Controllers, Models, Routes
Integration Testing	All 16 API endpoints verified	User, Property, Booking, Eco-Rating resources
Performance Testing	2 of 4 endpoints production-ready under load	4 suites: Auth, Properties, Bookings, Eco-Ratings

5.4 Recommendations for Future Improvement

- Implement server-side pagination on GET /api/properties to limit payload sizes and eliminate timeout errors under concurrent load.
- Introduce response caching (e.g., Redis) for frequently accessed endpoints such as property listings and pre-computed eco-rating scores.
- Refactor the eco-rating calculation service to use asynchronous background processing, storing results in the database and serving cached values on GET requests.
- Update Artillery test scenarios to include valid JWT token injection so that authenticated endpoints can be properly load tested for throughput under realistic conditions.
- Expand integration test coverage to include the authentication flow (login, registration, token refresh) and edge cases such as expired tokens and concurrent session handling.
- Integrate the test suite into a CI/CD pipeline (e.g., GitHub Actions) so that unit and integration tests are automatically executed on every commit and pull request.

5.5 Final Assessment

The GreenRent application has been demonstrated to be functionally correct and structurally well-tested. The unit and integration test results provide strong confidence in the reliability of all core features. Performance testing has successfully identified specific scalability constraints that, if addressed through the recommended optimisations, would prepare the application for production-level traffic. The testing process itself has been comprehensive, methodical, and well-documented, meeting the objectives established at the outset of this report.